ING2 GS1 G4

Conception d’une application bancaire

Modélisation et simulation applicative dans un contexte distribué

Mathis LANCE-RICHARDOT, Clément PREMOLI, Alice PARENT, Vincent DELB, Valérie TEXIER

Zaouche Djaouida
2025-2026

TABLE DES MATIÈRES

I - Vérification formelle et simulation : état de l'art..... 3
I.1 - Les réseaux de Pétri..... 3
I.2 - Modélisation fonctionnelle et simulation dans un contexte de concurrence avec Akka..... 5
II - Traduction vers un modèle formel..... 5
II. 1 Proposition de réseau de Pétri..... 5
II. 2. Expression des invariants et des propriétés structurelles..... 7
II. 2.1 Atomes propositionnels..... 8
II.2.2 Écriture des règles..... 9
II.2.3 Quelques conclusions et pistes d'amélioration..... 11
III - Simulation en temps réel..... 12
III.1 Architecture Akka..... 12
III.2 Présentation d'une simulation..... 14
Main :..... 14
Résultats :..... 15
Tests unitaires :..... 16
IV - Comparaison..... 17
Bibliographie, webographie et référence Github..... 18
Environnement de Développement (IDE)..... 18
Outils de Modélisation et Simulation..... 18
Travaux Académiques..... 18
Documentation Technique & Tutoriels..... 18
Ressources Audiovisuelles..... 18

Conception d’une application bancaire

Modélisation et simulation applicative dans un contexte distribué

Ce projet propose une approche conceptuelle d’application bancaire ainsi que son développement dans le langage Scala. Cette application doit permettre d’opérer des virements en ligne dans un contexte distribué et prendre en charge le respect des contraintes métiers inhérentes aux opérations qui encadrent la gestion de comptes. Les trois points critiques suivants sont identifiés, et constituent la ligne de mire qui sous-tendra tout le travail :

- le solde du compte émetteur doit être supérieur au montant du virement
- le virement doit être inférieur à un plafond défini (lutte contre la fraude)
- les opérations de crédit et de débit qui constituent les deux mouvements du virement ne peuvent exister l’une sans l’autre (propriété d’atomicité)

La robustesse de ce cadre sera évaluée à l’aune de formalismes de modélisation et de simulation que nous croiserons en préalable au travail d'implémentation.

I - Vérification formelle et simulation : état de l’art

I.1 - Les réseaux de Pétri

La vérification formelle constitue une approche méthodologique rigoureuse visant à assurer la fiabilité et la sécurité des applications dites critiques, telles que celles utilisées dans les domaines de l’aéronautique, du ferroviaire, médical ou encore bancaire. Reposant sur des fondements logico-mathématiques, cette démarche permet de démontrer, au moyen de preuves formelles, l’absence de défauts au sein d’un programme et la conformité totale de son comportement aux spécifications établies.

Contrairement aux méthodes de test classiques, limitées à un échantillon de cas d’exécution, la vérification formelle explore de manière exhaustive l’ensemble des scénarios possibles, garantissant ainsi un haut niveau de robustesse et de sûreté. Parmi les procédés les plus reconnus figurent la méthode B, qui s’appuie sur la modélisation et la preuve de propriétés à partir de spécifications formelles, ainsi que les analyses de circuits VLSI, employées pour vérifier la cohérence logique et la fiabilité des architectures matérielles complexes.

Dans la continuité de cette approche, la modélisation des systèmes distribués par les **réseaux de Pétri** offre un cadre théorique puissant pour représenter, analyser et valider les

comportements concurrents ou synchronisés de tels systèmes, ouvrant ainsi la voie à une vérification formelle adaptée aux environnements hautement parallèles.

Ce type de modélisation s'impose comme un outil formel particulièrement adapté à la représentation et à l'analyse du comportement des systèmes distribués et concurrents.

Fondés sur une structure bipartite composée de places aussi appelées états[1] et de transitions, reliées par des arcs orientés, les réseaux de Pétri permettent de décrire de manière explicite les relations de causalité, de synchronisation et de parallélisme entre les différentes composantes d'un système.

Grâce à leur formalisme mathématique, ils offrent la possibilité non seulement de modéliser le comportement dynamique d'un système - en termes d'événements et d'états -, mais également d'identifier ses propriétés structurelles, telles que la vivacité, l'absence de blocage ou la capacité de reprise. L'analyse des invariants, qu'ils soient de marquage ou de place, constitue en outre un moyen essentiel pour démontrer la préservation de ressources critiques ou la conformité à des contraintes de sûreté, renforçant ainsi la robustesse et la fiabilité globale du modèle.

La représentation en graphe d'événements et d'états permet l'exploration systématique de tous les comportements possibles du système modélisé. La construction de graphes de couverture joue un rôle central : elle permet de représenter l'ensemble des marquages atteignables et, par conséquent, de couvrir tous les chemins d'exécution possibles pour garantir une exhaustivité dans l'évaluation du modèle. Complétant cette approche structurelle, le calcul d'invariants de place ou de transition offrent une vue analytique des dépendances et des conservations internes du système.

En parallèle, l'intégration du formalisme de **logique temporelle linéaire** (LTL) étend la vérification en introduisant des règles logiques temporelles permettant de décrire et de démontrer la validité de propriétés dynamiques sur la séquence des états. Ce couplage entre les réseaux de Pétri et LTL offre ainsi un cadre unifié pour la spécification, la simulation et la preuve de correction des systèmes distribués, assurant une vérification formelle à la fois complète et expressive.

[1] Les deux termes se rencontreront dans le rapport, et seront employés de manière parfaitement synonymes.

I.2 - Modélisation fonctionnelle et simulation dans un contexte de concurrence avec Akka

En complément de l'approche comportementale dynamique des réseaux de Pétri, nous ferons également appel à des outils de simulation. Cette approche repose sur le principe fondamental d'acteurs qui interagissent selon les règles qui définissent le cadre critique et la robustesse applicative. Elle fait usage du **framework Akka** utilisé dans les contextes distribués des développements en Scala.

Typiquement, il s'agit de la construction d'une architecture sous la forme d'un système Akka / Scala qui permettra d'observer des échanges entre acteurs, des transitions et protocoles de communication. En outre, dans ces échanges, nous vérifierons les comportements en temps réel dans un contexte de concurrence. Plus généralement, un tel dispositif permet de superviser et réaliser des tests unitaires, vérifier la tolérance aux pannes, identifier les flux de messages critiques et les interactions…

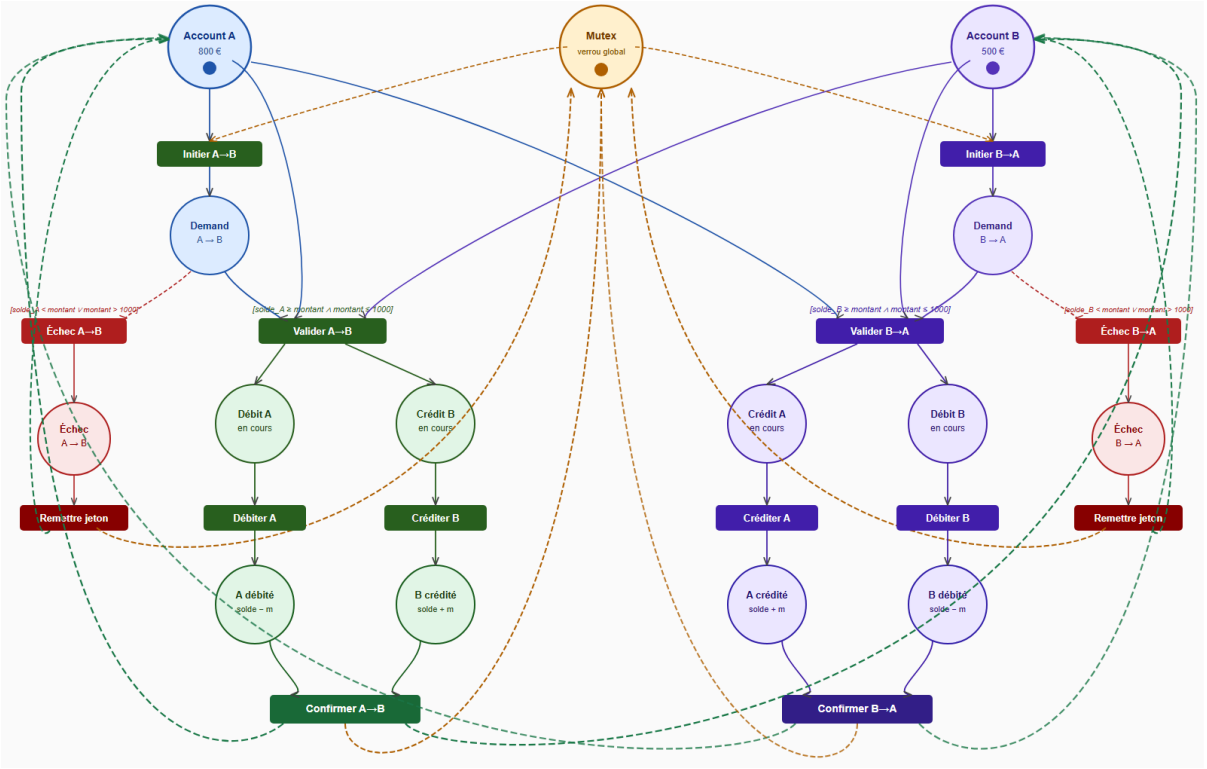
II - Traduction vers un modèle formel

Nous utilisons le formalisme des réseaux de Pétri pour capturer tous les chemins de communication de notre modèle au sein d'un espace d'états le plus exhaustif possible.
L'objectif est de concevoir et développer un analyseur pour l'étude des propriétés structurelles et des invariants métiers.

II. 1 Proposition de réseau de Pétri

La modélisation d'une transaction sécurisée entre deux comptes est un exercice intéressant qui nous a montré à quel point la criticité des opérations peut s'avérer fatale en cas de non-respect, et a poussé vers une conception qui maximise la sécurité dans les détails réglementaires. Néanmoins, pour des besoins de lisibilité dans le cadre de ce rendu, il a été nécessaire de positionner le curseur entre la recherche de la "perfection métier" et celle de "l'accessibilité informationnelle" à un niveau qui ne satisfait complètement ni l'une ni l'autre.

Nous présentons ci-dessous une version simple mais opérationnelle de notre réseau de Pétri à partir de laquelle nous dresserons les propriétés structurelles et les invariants critiques.



Légende :

(Un fichier .pnml du dépôt Github, exportable vers un simulateur en ligne, permet de vérifier le bon fonctionnement de ce réseau. Le lien du dépôt est donné en fin de rapport).

Principe général : Ce modèle représente une possibilité de virement entre un compte A et un compte B, doté chacun d'un solde à titre d'exemple : 800 euros pour A et 500 euros pour B. La circulation est rendue bijective, de A vers B et de B vers A par la construction d'une symétrie.

Le respect des règles critiques se matérialise par un principe de mutex ainsi que des transitions de validité. En outre, le plafond imposé par la lutte contre la fraude est fixé arbitrairement à 1000 € dans cet exemple.

Le jeton représente une demande de virement dont la position de départ est le compte. Son parcours transite par les étapes suivantes.

L'initialisation de la demande de virement par le compte émetteur passe par une première transition vers l'état *Demand*. C'est depuis cet état que la vérification des règles critiques, que deux matérialisent deux transitions *validation* et *échec*.

- La transition *validation* doit satisfaire la règle Solde_A ≥ montant Λ montant ≤ 1000
- La transition *échec* satisfait quant à elle la règle inverse : Solde_A < montant V montant > 1000
  (et pareil pour Solde_B de l'autre côté)

La transition *échec* a pour effet la remise du jeton à sa position de départ tandis que validation initie de manière parallèle le débit du compte émetteur et le crédit du compte récepteur, exprimant ainsi l'atomicité de ces deux opérations.

Les deux états signifiant l'opération en cours puis son achèvement, au moyen d'une transition d'action se rejoignent en une transition de confirmation, amenant à son tour au retour du jeton à sa position initiale.
Au centre de ces deux circuits symétriques de A → B et B → A se trouve un sous-circuit qui matérialise le principe d'un mutex. Celui-ci assure la séquentialité des accès aux ressources partagées que sont les soldes des comptes, essentiel aux opérations critiques en contexte distribué, en particulier à la vérification du solde avant tout transfert.

Celui-ci est doté d'un jeton en position de départ sur l'état mutex qui ne peut circuler que de manière exclusive vers l'initialisation de la demande, soit de A → B, soit de B → A.
Ainsi, pour qu'un virement se réalise, le sous-circuit concerné doit se trouver en possession de deux jetons : celui de son circuit et celui du mutex. Les transition d'échec et de confirmation constituent toutes deux des points de remise du mutex en position initiale afin de libérer la ressource.

II. 2. Expression des invariants et des propriétés structurelles

Comme nous l'avons vu, la modélisation selon le formalisme du réseau de Pétri donne une vision dans le temps du comportement dynamique souhaité, par le parcours d'un ou plusieurs jetons selon des règles de franchissement dans le circuit ainsi dessiné. Le formalisme permet de distinguer les propriétés comportementales observables des propriétés structurelles. Tandis que les premières s'attellent à montrer la vivacité du modèle, les secondes s'attachent à la consistance et la conservation.

L'expression de ces dernières s'appuie avantageusement sur la logique temporelle linéaire LTL qui permet de disposer du corpus des invariants critiques et propriétés structurelles, garantissant la validité du modèle sur l'espace d'états et le respect des contraintes métier. Elle permet d'observer l'évolution du système dans le temps (ce qui finit toujours par arriver, ou au contraire n'arrive jamais). Celle-ci repose sur des atomes propositionnels que nous articulons ensuite avec les opérateurs logiques du formalisme pour écrire les règles.

Elle permet également d'éviter les cas limites, c'est-à-dire toute situation qui met en danger la progression dans le réseau d'un état vers un autre (deadlock) notamment consécutive à la violation des règles d'atomicité (risque de transfert partiel…) ou d'exclusion mutuelle.

Le principe de progression est celui qui garantit qu'à partir de tout état atteignable, il doit toujours être possible de faire un pas (continuer l'exécution, ou bien boucler proprement).
Pour cela, LTL définit des atomes propositionnels qui prennent une valeur vraie s'il existe, depuis un état non-terminal, au moins une transition tirable (ou, plus finement, si le système n'est pas bloqué dans un marquage mort).

II. 2.1 Atomes propositionnels

Le tableau ci-dessous reprend les atomes propositionnels dont nous aurons besoin, comme briques de base.

Libellé	Signification
`SoldeA_ge_Montant`	Le solde du compte A est supérieur ou égal au montant du transfert A→B.
`SoldeB_ge_Montant`	Le solde du compte B est supérieur ou égal au montant du transfert B→A.
`MontantA2B_le_1000`	Le montant du transfert A→B est inférieur ou égal à 1000 €, plafond de lutte contre la fraude.
`MontantB2A_le_1000`	Le montant du transfert B→A est inférieur ou égal à 1000 €, plafond de lutte contre la fraude.
`InitialiserA2B_tire`	Situation d'une demande de virement de A→B initialisée.
`InitialiserB2A_tire`	Situation d'une demande de virement de B→A initialisée.
`DemandAtoB`	Etat du compte A après la demande d'initialisation
`DemandBtoA`	Etat du compte B après la demande d'initialisation
`DebiterA`	Action de débit du compte A (transition « Débit A » tirée, ou état « A débité » atteint).
`DebiterB`	Action de débit du compte B (transition « Débit B » tirée, ou état « B débité » atteint).
`EchecA2B_tire`	Situation d'une d'un virement de A→B non réalisé car échoué.
`EchecB2A_tire`	Situation d'une d'un virement de B→A non réalisé car échoué.
`CrediterA`	Action de crédit du compte A (transition « Crédit A » tirée, ou état « A crédité » atteint).
`CrediterB`	Action de crédit du compte B (transition « Crédit B » tirée, ou état

	« B crédité » atteint).
`TransfertA2B_en_cours`	Situation dans une phase de transfert A→B Marquage dans la branche A→B, entre « Demande A→B » et « Confirmer A→B » fait .
`TransfertB2A_en_cours`	Situation dans une phase de transfert B→A Marquage dans la branche B→A, entre « Demande B→A » et « Confirmer B→A ».
`A_debite`	Situation du compte A débité. Représente l'état « A débité » atteint dans le réseau.
`B_debite`	Situation du compte B débité. Représente l'état « B débité » atteint dans le réseau.
`A_credite`	Situation du compte A crédité. Représente l'état « A crédité » atteint dans le réseau.
`B_credite`	Situation du compte B crédité. Représente l'état « B crédité » atteint dans le réseau.
`A2B_fait`	Situation de virement A→B confirmé.
`B2A_fait`	Situation de virement B→A confirmé.
`MutexOccupe`	Le jeton du mutex n'est plus dans la place « Mutex », il est acquis par une des branches.

II.2.2 Écriture des règles

Sur un réseau de Pétri, l'analyse structurelle permet de calculer des :

- P-invariants (invariants de places)
- T-invariants (invariants de transitions)

Ces invariants expriment des conservations ou des circuits d'exécution complets. Ils donnent des propriétés structurelles qui fournissent des garanties de conservation, d'absence de dépassement de bornes, ou d'existence de cycles complets de virements (en l'espèce, mutex toujours à un jeton, masse monétaire constante, complétude des cycles de transfert). Ces invariants sont des propriétés inductives qui, vraies au marquage initial, le restent pour tous les marquages atteignables. Ils soutiennent l'écriture des règles par assemblage des atomes propositionnels. Les symboles du formalisme LTL sont rappelés ci-dessous :

- □ : Toujours, globalement
- ◇ : Un jour, éventuellement, si besoin, futur
- ¬ : Négation
- ∧, ∨, ⇒ : conjonction, disjonction, implication

La bidirectionnalité des virements dans ce réseau impose le dédoublement des règles pour tenir compte de la circulation des jetons dans les deux branches du modèle.

Règle 1 : Solde du compte émetteur toujours suffisant
Lorsqu'un transfert A→B est demandé, le solde de A doit rester suffisant, et réciproquement.

```
□ G(DemandA2B ⇒ SoldeA_ge_Montant v ◇(EchecA2B_tire) )
□ G(DemandB2A ⇒ SoldeB_ge_Montant v ◇(EchecB2A_tire) )
```

Règle 2 : Transfert borné au plafond de 1000 €
Le montant des virements doit rester inférieur à 1000 €, de façon à limiter les transferts illégaux (blanchiment, fraude…)

```
□ ( TransfertA2B_en_cours ⇒ MontantA2B_le_1000 )
□ ( TransfertB2A_en_cours ⇒ MontantB2A_le_1000 )
```

Règle 3 : couplage débit-crédit
Il s'agit ici de vérifier la propriété d'atomicité de l'opération de transfert, de manière à rendre solidaire le débit d'une part et le crédit d'autre part.
Cette règle peut s'écrire de 2 manières :

- sous la forme d'une contrainte de corrélation immédiate sur les états de transfert en cours

```
□ ( DebiterA ⇒ ◇ CrediterB )
□ ( CrediterB ⇒ ◇ DebiterA )
□ ( DebiterB ⇒ ◇ CrediterA )
□ ( CrediterA ⇒ ◇ DebiterB )
```

- ou bien de manière plus globale, sur les états finaux

```
□ ( A2B_fait ⇒ (A_debite ∧ B_credite))
□ ( B2A_fait ⇒ (B_debite ∧ A_credite))
```

Règle 4 : Règle d'exclusion mutuelle du mutex

Cette règle est d'intérêt plus technique que métier, mais tout aussi importante pour le maintien de l'intégrité du système. Le sous-circuit du mutex ne contient qu'un seul jeton qui pourra circuler alternativement dans l'une ou l'autre des branches de virement selon sa direction, de sorte que la section critique (transfert en cours) ne contienne au plus qu'un seul transfert.

La transition d'initialisation du virement ne peut « tirer » que si elle fait l'acquisition du jeton mutex depuis sa place mutex initiale. De leurs côtés, en sortie de circuit, les transitions échec et validation redonnent le jeton au mutex afin de libérer la ressource.

Cette exclusion mutuelle interdit la circulation de deux transferts en parallèle. L'expression LTL de cette règle peut s'écrire ainsi soit au regard de l'opération en cours, soit au regard du statut du mutex.

- Interdiction de deux transferts simultanés entre les deux branches.

```
□ ¬( TransfertA2B_en_cours ∧ TransfertB2A_en_cours )
```

- Considération de l'occupation ou de la libération du mutex. On utilise une variable X pour représenter l'opérateur « Next » (état suivant).
    - Quand un transfert est initialisé le mutex devient occupé.

```
□ ( InitialiserA2B_tire ⇒ X MutexOccupe )
□ ( InitialiserB2A_tire ⇒ X MutexOccupe )
```

    - Quand un transfert est soit confirmé, soit en échec, le mutex est libéré.

```
□ ( A2B_fait ⇒ X ¬MutexOccupe )
□ ( B2A_fait ⇒ X ¬MutexOccupe )
□ ( EchecA2B_tire ⇒ X ¬MutexOccupe )
□ ( EchecB2A_tire ⇒ X ¬MutexOccupe )
```

II.2.3 Quelques conclusions et pistes d'amélioration

Le principe de progression se vérifie s'il est possible de formuler des LTL de réversibilité :

```
□ ( vrai ⇒ F (CompteActifA ∧ CompteActifB ∧ ¬MutexOccupe) )
```

avec *CompteActifA* et *CompteActifB* valant états initiaux des deux branches du réseau.
Cela exprime que depuis tout état atteignable, on peut revenir à une configuration « saine » où les deux comptes sont actifs et le mutex est disponible.

La complétude des cycles de virements est quant à elle exprimée par des T-invariants qui se déduisent au moyen de séquences de transitions ramenant le réseau à un marquage identique (ou équivalent) à l'initial. Dans notre cas, un cycle complet A→B (ou B→A) peut former un T-invariant. Pour A→B, par exemple :

```
1.  T1 (Initialiser A->B)
2.  T2 (Valider A->B)
3.  T3 (Débiter A)
4.  T4 (Créditer B)
5.  T5 (Confirmer A->B)
```

L'ensemble de transitions {T1, T2, T3, T4, T5} correspond à une exécution complète de la transaction A→B, qui ramène le système (compte actif A, compte actif B, mutex) dans

une configuration équivalente au départ. Cela permet d'argumenter que tout début de transfert correctement lancé peut, structurellement, se terminer.

Plusieurs pistes sont possibles pour améliorer notre réseau. L'une d'elles pourrait proposer un découpage plus précis des cas d'échec, en associant non plus une globale mais deux places locales *échec*. La première liée à la règle du solde suffisant, et l'autre au plafond de virement. Une autre pourrait consister en l'ajout des places représentant le solde. Avec cette modification, d'autres P-invariants pourraient être obtenus tels que, pour des places {SoldeA, A_debite, A_credité, etc.}. Cela permet d'exprimer la conservation de la masse monétaire du compte A vue à travers les différentes phases de la transaction (avant, pendant, après).

De même, la modélisation explicite des soldes, avec des places comme SoldeA et SoldeB permettrait la définition d'un invariant global de type :

```
m(SoldeA) + m(SoldeB) = somme_totale_constante
```

ce qui traduit l'idée que l'argent ne disparaît pas, il ne fait que changer de compte.

III - Simulation en temps réel

L'objectif de cette section est de traduire les propriétés formelles identifiées dans la partie II en une architecture logicielle concrète, permettant de confronter le modèle théorique à une exécution réelle en environnement distribué.

III.1 Architecture Akka

Le développement de l'application vise à simuler le comportement réel du système bancaire via le framework Akka / Scala. Cette architecture est conçue pour permettre l'observation des échanges entre acteurs, des transitions et des protocoles de communication. Le framework Akka permet de redémarrer les acteurs individuellement en cas de crash et d'éviter les problèmes de communication entre acteurs (deadlock). L'objectif est de valider le modèle théorique en vérifiant le comportement du système dans un contexte de concurrence.

Ce dispositif permet concrètement de :

- Superviser et réaliser des tests unitaires.
- Vérifier la tolérance aux pannes.
- Identifier les flux de messages critiques et les interactions entre les composants du système distribué.

Le système repose sur trois acteurs qui séparent les préoccupations : la gestion de l'état, la journalisation (traçabilité) et la coordination des transactions. Les acteurs sont tous typés, c'est-à-dire qu'on définit quel type de message ils reçoivent. Typer les acteurs simplifie la conception, permettant de voir plus nettement où est le problème de communication s'il y a des soucis à la compilation. Dans le cas des acteurs Akka classiques (non typés), en

utilisant la fonction Receive, on va retourner une valeur Any (n'importe quelle valeur). Ce sera donc plus compliqué de trouver s'il y a une erreur.

- BankAccount.scala : la gestion de l'état, typé BankMessage

  C'est l'acteur le plus simple, mais aussi le plus critique :

  - Rôle : Il encapsule l'état financier d'un compte (le solde).
  - Responsabilité : Il applique les règles métier (par exemple, le plafond de retrait)
  - Fonctionnement : Il ne modifie jamais son état de manière externe. Lorsqu'il reçoit un message (Débit, Crédit), il vérifie les conditions, notifie le Ledger et crée un nouveau comportement (active) avec le nouveau solde. C'est l'illustration parfaite de l'état immuable.

- Ledger.scala : la mémoire du système, typé LedgerMessage

  - Rôle : Il agit comme un journal d'audit centralisé.
  - Responsabilité : Enregistrer toutes les transactions et répondre aux requêtes de reporting.
  - Spécificité technique : L'utilisation de collectBalances est intéressante ici. C'est un pattern de "Scatter-Gather" : il délègue la collecte des soldes à un acteur temporaire ("spawnAnonymous") pour agréger les résultats de manière asynchrone avant de les afficher.

- TransferManager.scala : le coordinateur de transactions, typé TransferMessage

  - Rôle : Gérer la complexité des transactions multi-acteurs (Atomicité).
  - Responsabilité : S'assurer qu'un transfert est complet (Débit d'un côté + Crédit de l'autre).
  - Mécanisme clé : Il implémente une machine à états finis (FSM). Il passe par plusieurs étapes :
    - idle : attente d'une requête
    - waitingDebit : vérifie si le retrait est autorisé et crée une file d'attente des transferts
    - waitingCredit : confirme le dépôt
  - Gestion des erreurs : Grâce à l'utilisation de messageAdapter, le manager traduit les réponses des comptes en messages compréhensibles, permettant une gestion élégante des succès et des échecs sans bloquer le système.

III.2 Présentation d'une simulation

Main :

```
    // — Supervision : restart automatique en cas d'exception ————————————
    val supervisedAccount = (id: String, balance: Double) =>
      Behaviors
        .supervise(BankAccount(id, balance, ledger))
        .onFailure[Exception](SupervisorStrategy.restart)

    val alice   = context.spawn(supervisedAccount("Alice",   800.0), "Alice")
    val bob     = context.spawn(supervisedAccount("Bob",     500.0), "Bob")
    val charlie = context.spawn(supervisedAccount("Charlie", 300.0), "Charlie")

    ledger ! RegisterAccount("Alice",   alice)
    ledger ! RegisterAccount("Bob",     bob)
    ledger ! RegisterAccount("Charlie", charlie)

    // — Supervision du TransferManager : resume en cas d'erreur non critique —
    val supervisedTransfer =
      Behaviors
        .supervise(TransferManager(ledger))
        .onFailure[Exception](SupervisorStrategy.resume)

    val transfer = context.spawn(supervisedTransfer, "TransferManager")

    transfer ! TransferRequest(alice,   bob,     200.0)
    transfer ! TransferRequest(bob,     charlie, 150.0)
    transfer ! TransferRequest(charlie, alice,   500.0)
    transfer ! TransferRequest(alice,   bob,    1500.0)

    context.scheduleOnce(2.seconds, ledger, PrintHistory)
    context.scheduleOnce(3.seconds, ledger, PrintBalances)

    Behaviors.empty
  }

  val system = ActorSystem[Nothing](root, "BankSystem")
  Thread.sleep(4000)
  system.terminate()
}
```

Initialisation avec des superviseurs qui permettent de redémarrer l'acteur en cas de crash

Résultats :

```
[info] welcome to sbt 1.12.6 (Oracle Corporation Java 24.0.1)
[info] loading project definition from C:\Users\Stigeon\IdeaProjects\Project_Bank\project
[info] loading settings for project root from build.sbt...
[info] set current project to Project_Bank (in build file:/C:/Users/Stigeon/IdeaProjects/Project_Bank/)
[info] compiling 5 Scala sources to C:\Users\Stigeon\IdeaProjects\Project_Bank\target\scala-3.3.7\classes ...
[info] running Bank.Main

============== HISTORIQUE GLOBAL ==============
  1. DEMANDE  | Alice -> Bob | montant=200.0
  2. DEBIT    | Alice | montant=200.0 | nouveau solde=600.0
  3. DEBIT_OK | Alice -> Bob | montant=200.0
  4. CREDIT   | Bob | montant=200.0 | nouveau solde=700.0
  5. CONFIRME | Alice -> Bob | montant=200.0
  6. DEMANDE  | Bob -> Charlie | montant=150.0
  7. DEBIT    | Bob | montant=150.0 | nouveau solde=550.0
  8. DEBIT_OK | Bob -> Charlie | montant=150.0
  9. CREDIT   | Charlie | montant=150.0 | nouveau solde=450.0
  10. CONFIRME | Bob -> Charlie | montant=150.0
  11. DEMANDE  | Charlie -> Alice | montant=500.0
  12. ECHEC_DEBIT | Charlie | montant=500.0 | raison= solde insuffisant
  13. ECHEC    | Charlie -> Alice | montant=500.0
  14. DEMANDE  | Alice -> Bob | montant=1500.0
  15. ECHEC_DEBIT | Alice | montant=1500.0 | raison= plafond depasse
  16. ECHEC    | Alice -> Bob | montant=1500.0
===============================================

============== SOLDES FINAUX ==================
  Alice : 600.0 EUR
  Bob : 550.0 EUR
  Charlie : 450.0 EUR
===============================================

[success] Total time: 12 s, completed 13 avr. 2026, 17:18:37
```

3. Logs

```
Systeme demarre avec 3 comptes
Compte cree : Compte1 | solde initial = 500.0 EUR
Compte cree : Compte2 | solde initial = 500.0 EUR
Compte cree : Compte3 | solde initial = 500.0 EUR
>> Virement demande : Compte1 -> Compte2 | 200.0 EUR
DEMANDE  | Compte1 -> Compte2 | montant=200.0
DEBIT    | Compte1 | montant=200.0 | nouveau solde=300.0
DEBIT_OK | Compte1 -> Compte2 | montant=200.0
CREDIT   | Compte2 | montant=200.0 | nouveau solde=700.0
CONFIRME | Compte1 -> Compte2 | montant=200.0
=== soldes après virement ====================
  Compte1      : 300,0 EUR
  Compte2      : 700,0 EUR
===============================================
```

Flux simplifié : on ne détaille pas les entrées et sorties de la file d'attente des transferts. De plus, on n'a pas à gérer des états partagés.

On définit le format des messages avec ce qui ressemble, conceptuellement, à un “header” global : Bank.models. De plus, “sealed trait” sert de “header” de classification et permet de définir la structure et la catégorie du message.

Tests unitaires :

```
SLF4J(W): A number (3) of logging calls during the initialization phase have been intercepted and are
SLF4J(W): now being replayed. These are subject to the filtering rules of the underlying logging system.
SLF4J(W): See also https://www.slf4j.org/codes.html#replay
[info] BankAccountSpec:
[info] BankAccount
[info] - should accepter un debit valide et retourner DebitSuccess
[info] - should refuser un debit si solde insuffisant et retourner DebitFailure
[info] - should refuser un debit depassant le plafond de 1000 EUR et retourner DebitFailure
[info] - should accepter un credit et retourner CreditSuccess
[info] - should maintenir un solde coherent apres plusieurs operations sequentielles
[info] - should retourner le solde correct via GetBalance
[info] TransferManagerSpec:
[info] TransferManager
[info] - should effectuer un transfert valide entre deux comptes
[info] - should ne pas modifier les soldes si le debit echoue
[info] - should ne pas modifier les soldes si le montant depasse le plafond
[info] - should traiter plusieurs transferts sequentiellement dans l'ordre
[info] Run completed in 1 second, 959 milliseconds.
[info] Total number of tests run: 10
[info] Suites: completed 2, aborted 0
[info] Tests: succeeded 10, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 10 s, completed 13 avr. 2026, 17:40:35
```

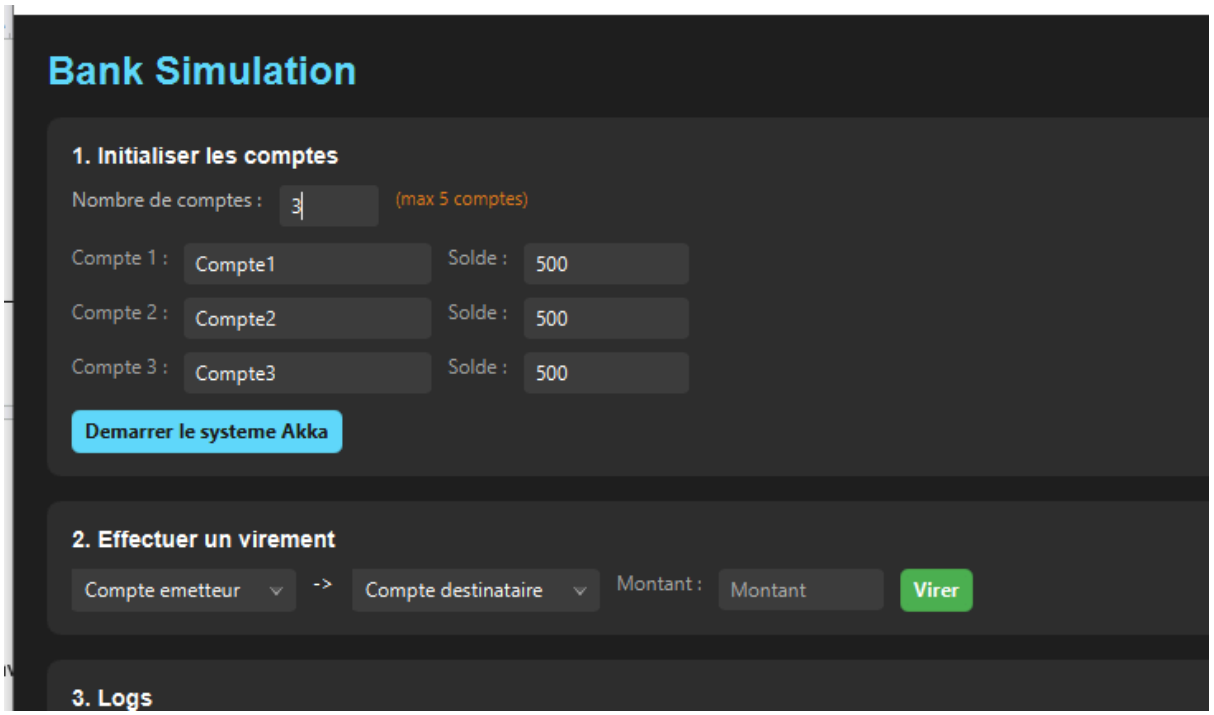
Les tests couvrent tous les chemins du système. Comme tous les tests réussissent, on peut affirmer que l’implémentation de l’application bancaire est robuste.

IV - Comparaison

La simulation montre le respect des arcs du réseau de Pétri via les messages envoyés aux différents acteurs. Les invariants métier sont donc conservés.
En programmation, l'ordre des lignes de code permet d'exécuter pas à pas les instructions qu'il s'agit donc de lancer dans le bon ordre pour garantir l'intégrité vis à vis des règles. C'est précisément le rôle du mutex dans le réseau qui est repris ici.

Le lancement d'une transition dans le réseau de Pétri se fait aléatoirement, tandis que dans une simulation, il est possible de définir le chemin à emprunter. La modélisation est donc bien reprise dès lors que la définition des chemins et des invariants est respectée à l'exécution : l'intégrité transactionnelle, la vivacité et la progression (absence de deadlock) est observée de manière identique.

De plus, la simulation permet de complexifier le réseau de Petri en ajoutant des comptes sans effort : la généralisation du cheminement de message a lieu sans duplication de place ou de transition comme on le ferait dans notre réseau de Petri.



Bibliographie, webographie et référence Github

Les références ci-dessous permettent de cadrer théoriquement et méthodologiquement ce projet. Le lien Github est donné en fin de liste.

Environnement de Développement (IDE)

- **JetBrains.** *IntelliJ IDEA : L'IDE de référence pour le développement Scala et Java*. [Logiciel]. Comprend le support natif de SBT (Scala Build Tool) et l'intégration des frameworks de concurrence. Disponible sur : https://www.jetbrains.com/idea/

Outils de Modélisation et Simulation

- **Petri Net Editor (PES).** *Outil de modélisation et d'analyse de réseaux de Petri en ligne*. Université technique d'Ostrava (VŠB). Disponible sur : https://pes.vsb.cz/petrineteditor/

Travaux Académiques

- **Delfieu, D., Roux, O. H., & Valette, R.** (s.d.). *Réseau de Pétri : Théorie et mise en œuvre*. HAL Open Science. Consulté à l'adresse : https://hal.science/hal-05069404v1/file/main_ti.pdf
- **Laboratoire de Méthodes Formelles (LMF), CNRS.** *Archives et publications de Serge Haddad et François Vernadat*. [En ligne]. Disponible sur : https://home.lmf.cnrs.fr/downloads/SergeHaddad/

Documentation Technique & Tutoriels

- **Scala Documentation.** *Documentation de référence du langage Scala*. [En ligne]. Disponible sur : https://docs.scala-lang.org/
- **Baeldung.** *Akka Series: A Comprehensive Guide to Akka with Scala*. [Tutoriel en ligne]. Disponible sur : https://www.baeldung.com/scala/akka-series

Ressources Audiovisuelles

- **Bogard, G.** (2014). *Introduction aux acteurs Akka : concurrence et systèmes distribués pour la JVM*. Conférence présentée à ScalaIO FR. [Vidéo]. YouTube. https://www.youtube.com/watch?v=tcj8IaG9_wc

Lien Github :
https://github.com/MathisLanceRichardot/Project_Bank